

ASSIGNMENT 15.2

K. Keerthana
2303A52027
Batch-38

Task Descriptor-1: (Setup Fast API Backend)

Instructions:

- Install FastAPI and Uvicorn.
- Use AI to generate a basic FastAPI application with one endpoint

Sample Code:

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}
```

Expected Output:

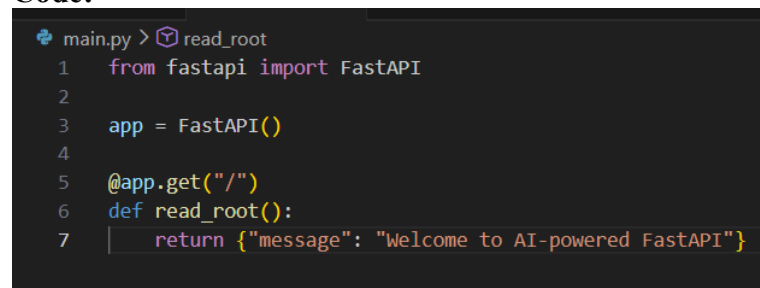
Running the server and accessing:

http://127.0.0.1:8000/

Response:

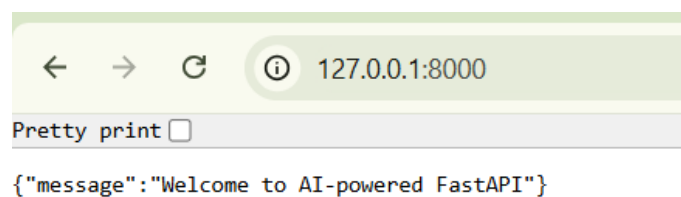
```
{"message": "Welcome to AI-powered FastAPI"}
```

Code:

A screenshot of a code editor with a dark background. The code is written in Python and defines a FastAPI application. The code is as follows:

```
main.py > read_root
1  from fastapi import FastAPI
2
3  app = FastAPI()
4
5  @app.get("/")
6  def read_root():
7      return {"message": "Welcome to AI-powered FastAPI"}
```

Output:

A screenshot of a web browser window. The address bar shows the URL '127.0.0.1:8000'. Below the address bar, there is a 'Pretty print' checkbox which is currently unchecked. The response body shows the JSON output:

```
{"message": "Welcome to AI-powered FastAPI"}
```

default

GET / Read Root

Parameters Try it out

No parameters

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/
```

Server response

Code Details

200

Response body

```
{
  "message": "Welcome to AI-powered FastAPI"
}
```

Response headers

```
content-length: 43
content-type: application/json
date: Mon, 16 Mar 2026 15:12:17 GMT
server: uvicorn
```

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/
```

Server response

Code Details

200

Response body

```
{
  "message": "Welcome to AI-powered FastAPI"
}
```

Response headers

```
content-length: 43
content-type: application/json
date: Mon, 16 Mar 2026 15:12:17 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
"string"
```

Task Descriptor- 2: (API – Create and Read Items)

Instructions:

- Implement endpoints to add and retrieve products.
- Use a Python list as temporary storage.

Sample Code:

```
items = []
@app.get("/items")
def get_items():
```

```
return items
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
```

Expected Output:

GET /items → []

POST /items with {"name":"Pen","price":20}

→ {"message":"Item created","item":{"name":"Pen","price":20}}

Code:

```
main.py > get_items
1 from fastapi import FastAPI
2 app = FastAPI()
3 # temporary storage
4 items = []
5 @app.get("/")
6 def read_root():
7     return {"message": "Welcome to AI-powered FastAPI"}
8 @app.get("/items")
9 def get_items():
10    return items
11 @app.post("/items")
12 def add_item(item: dict):
13     items.append(item)
14     return {"message": "Item created", "item": item}
```

Output:

FastAPI 0.1.0 OAS 3.1
/openapi.json

default

GET / Read Root

GET /items Get Items

POST /items Add Item

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "name": "pen",
  "price": 20
}
```

Execute Clear

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8080/items' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Pen",
    "price": 20
  }'
```

Request URL

<http://127.0.0.1:8080/items>

Server response

Code	Details
200	Response body <pre>{ "message": "Item created", "item": { "name": "Pen", "price": 20 } }</pre> Response headers <pre>content-length: 59 content-type: application/json date: Mon, 16 Mar 2025 15:28:25 GMT server: unicorn</pre>

Responses

Code	Description	Links
200	Successful Response Media type: application/json Controls Accept header. Example Value Schema <pre>"string"</pre>	No links
422	Validation Error Media type: application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	No links

Task Descriptor- 3: (Update Existing Item)

Instructions:

- Use AI to generate an endpoint to update an item using its index.

Sample Code:

```
@app.put("/items/{index}")
```

```
def update_item(index: int, item: dict):
```

```
if index >= len(items):
```

```
return {"error": "Item not found"}
```

```
items[index] = item
```

```
return {"message": "Item updated", "item": item}
```

Expected Output:

```
PUT /items/0 with {"name":"Pencil","price":10}
```

```
→ {"message":"Item updated","item":{"name":"Pencil","price":10}}
```

Code:

```
Welcome  main.py 1 ●
main.py > update_item
1  from fastapi import FastAPI
2
3  app = FastAPI()
4
5  # temporary storage
6  items = []
7
8  # root endpoint
9  @app.get("/")
10 def read_root():
11     return {"message": "Welcome to AI-powered FastAPI"}
12
13 # get all items
14 @app.get("/items")
15 def get_items():
16     return items
17
18 # add item
19 @app.post("/items")
20 def add_item(item: dict):
21     items.append(item)
22     return {"message": "Item created", "item": item}
23
```

```
17
18 # add item
19 @app.post("/items")
20 def add_item(item: dict):
21     items.append(item)
22     return {"message": "Item created", "item": item}
23
24 # update item using index
25 @app.put("/items/{index}")
26 def update_item(index: int, item: dict):
27     if index >= len(items):
28         return {"error": "Item not found"}
29
30     items[index] = item
31     return {"message": "Item updated", "item": item}
```

Output:

FastAPI

0.1.0

OAS 3.1

/openapi.json

default

GET / Read Root

GET /items Get Items

POST /items Add Item

PUT /items/{index} Update Item

Schemas

HTTPValidationError > Expand all object

ValidationError > Expand all object

POST /items Add Item

Parameters

No parameters

Request body required application/json

Edit Value Schema

```
{  "name": "Pen",  "price": 20}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  http://127.0.0.1:8000/items' \  -H 'accept: application/json' \  -H 'content-type: application/json' \  -d '{  "name": "Pen",  "price": 20}'
```

Request URL

http://127.0.0.1:8000/items

Server response

Code Details

200

Response body

```
{  "message": "Item created",  "item": {    "name": "Pen",    "price": 20  } }
```

Response headers

```
content-length: 59  content-type: application/json  date: Mon, 16 Mar 2020 15:37:53 GMT  server: uvicorn
```

Responses

Code Description Links

200 Successful Response

Media type

application/json

Controls Accept header

Example Value Schema

No links

Responses		
Code	Description	Links
200	Successful Response	No links
Media type application/json		
Controls Accept header.		
Example Value Schema		
"string"		
422	Validation Error	No links
Media type application/json		
Example Value Schema		
{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }		

PUT

/items/{index} Update Item

⌵

Parameters

Cancel

Reset

Name	Description
Index required	
integer (path)	0

Request body required

application/json

Edit Value | Schema

{
 "name": "Pencil",
 "price": 10
}

Execute

Clear

Responses

Curl

curl -X 'PUT' \
 http://127.0.0.1:8000/items/0 \
 -H 'accept: application/json' \
 -H 'Content-Type: application/json' \
 -d '{
 "name": "Pencil",
 "price": 10
 }'

Request URL

http://127.0.0.1:8000/items/0

Server response

Code	Details
200	Response body { "message": "Item updated", "item": { "name": "Pencil", "price": 10 } } Response headers content-length: 62 content-type: application/json date: Mon, 16 Mar 2026 15:39:57 GMT server: uvicorn

Responses

Code	Description	Links
200	Successful Response	No links
Media type application/json		
Controls Accept header.		

Responses		
Code	Description	Links
200	Successful Response	No links
	Media type application/json Controls Accept header Example Value Schema "string"	
422	Validation Error	No links
	Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	

Task Descriptor- 4: (Delete Item)

Instructions:

- Implement deletion of an item by index.

Sample Code:

```
@app.delete("/items/{index}")
def delete_item(index: int):
    if index >= len(items):
        return {"error": "Item not found"}
    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}
```

Expected Output:

DELETE /items/0

→ {"message": "Item removed", "item": {"name": "Pencil", "price": 10}}

Code:

```

Welcome  main.py 1 X
main.py > delete_item
23 # update item
24 @app.put("/items/{index}")
25 def update_item(index: int, item: dict):
26     if index >= len(items):
27         return {"error": "Item not found"}
28
29     items[index] = item
30     return {"message": "Item updated", "item": item}
31
32 # delete item
33 @app.delete("/items/{index}")
34 def delete_item(index: int):
35     if index >= len(items):
36         return {"error": "Item not found"}
37
38     removed = items.pop(index)
39     return {"message": "Item removed", "item": removed}

```


Output:

```
Welcome  main.py 1 X
main.py > delete_item
23 # update item
24 @app.put("/items/{index}")
25 def update_item(index: int, item: dict):
26     if index >= len(items):
27         return {"error": "Item not found"}
28
29     items[index] = item
30     return {"message": "Item updated", "item": item}
31
32 # delete item
33 @app.delete("/items/{index}")
34 def delete_item(index: int):
35     if index >= len(items):
36         return {"error": "Item not found"}
37
38     removed = items.pop(index)
39     return {"message": "Item removed", "item": removed}
```

FastAPI 0.1.0 OAS 3.1
/openapi.json

default

- GET** / Read Root
- GET** /items Get Items
- POST** /items Add Item
- PUT** /items/{index} Update Item
- DELETE** /items/{index} Delete Item

POST /items Add Item

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "name": "Pencil",
  "price": 10
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/items' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Pencil",
    "price": 10
  }'
```

Request URL

http://127.0.0.1:8000/items

Server response

Code	Details
200	Response body <pre>{ "message": "Item created", "item": { "name": "Pencil", "price": 10 } }</pre> Download Response headers <pre>content-length: 62 content-type: application/json date: Mon, 16 Mar 2026 15:48:22 GMT server: uvicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept Header Example Value Schema <pre>"string"</pre>	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	No links

DELETE

/items/{index}

Delete Item

Parameters

Name	Description
index required	
integer	0
(path)	

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/items/0' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/items/0

Server response

Code	Details
------	---------

Responses

Curl

```
curl -X 'DELETE' \
'http://127.0.0.1:8000/items/0' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/items/0
```

Server response

Code	Details
200	Response body <pre>{ "message": "Item removed", "item": { "name": "Pencil", "price": 10 } }</pre> Response headers <pre>content-length: 62 content-type: application/json date: Mon, 16 Mar 2026 15:49:04 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Responses

Code	Description	Links
200	Successful Response Media type: application/json Controls Accept header. Example Value Schema <pre>"string"</pre>	No links
422	Validation Error Media type: application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	No links

Task Descriptor -5: (Add Auto-Generated Documentation)

Instructions:

- Use AI to add inline comments and docstrings for all endpoints.
- Optionally, integrate Swagger / Flask-RESTX to auto-generate API documentation.

Sample Code:

```
@app.route('/items', methods=['GET'])
def get_items():
    """
```

GET /items

Returns a list of all items in the store.

Code:

```
main.py > get_items
1  from fastapi import FastAPI
2  # Create FastAPI application
3  app = FastAPI(
4      title="Product Management API",
5      description="A simple FastAPI CRUD application to manage store items",
6      version="1.0"
7  )
8  # Temporary storage for items
9  items = []
10 @app.get("/")
11 def read_root():
12     """
13     GET /
14
15     Returns a welcome message for the API.
16     """
17     return {"message": "Welcome to AI-powered FastAPI"}
18 @app.get("/items")
19 def get_items():
20     """
21     GET /items
22
23     Returns a list of all items stored in memory.
24     """
25     return items
26 @app.post("/items")
27 def add_item(item: dict):
28     """
29     POST /items
30
31     Adds a new item to the store.
32
```

```
main.py > get_items
26 @app.post("/items")
27 def add_item(item: dict):
28     """
29     POST /items
30
31     Adds a new item to the store.
32
33     Parameters:
34     item (dict): JSON object containing item details such as name and price.
35
36     Returns:
37     JSON message confirming item creation.
38     """
39     items.append(item)
40     return {"message": "Item created", "item": item}
41
42
43 @app.put("/items/{index}")
44 def update_item(index: int, item: dict):
45     """
46     PUT /items/{index}
47
48     Updates an existing item using its index.
49
50     Parameters:
51     index (int): Position of the item in the list.
52     item (dict): Updated item data.
53
54     Returns:
55     JSON message confirming item update.
56     """
57     if index >= len(items):
```

```

Welcome  main.py 1
main.py > get_items
44 def update_item(index: int, item: dict):
45     if index >= len(items):
46         return {"error": "Item not found"}
47
48     items[index] = item
49     return {"message": "Item updated", "item": item}
50
51
52 @app.delete("/items/{index}")
53 def delete_item(index: int):
54     """
55     DELETE /items/{index}
56
57     Deletes an item from the list using its index.
58
59     Parameters:
60     index (int): Position of the item to remove.
61
62     Returns:
63     JSON message confirming deletion.
64     """
65     if index >= len(items):
66         return {"error": "Item not found"}
67
68     removed = items.pop(index)
69     return {"message": "Item removed", "item": removed}
70
71
72
73
74
75
76
77
78
79
80
81
82
83
```

Output:

FastAPI 0.1.0 OAS 3.1
/openapi.json

default

GET	/	Read Root	⌵
GET	/items	Get Items	⌵
POST	/items	Add Item	⌵
PUT	/items/{index}	Update Item	⌵
DELETE	/items/{index}	Delete Item	⌵

Schemas

HTTPValidationError > Expand all object

ValidationError > Expand all object